تمرین هفته دوم: سوپرمارکت زنده

در هفته اول صفحه سوپرمارکت خود را فقط با HTML و CSS ساختید و همه محصولات، قیمت‌ها و دسته‌ها را دستی داخل HTML نوشتید. در یک فروشگاه واقعی این اطلاعات از سرور می‌آید: قیمت عوض می‌شود، کالا تمام می‌شود و تخفیف جدید اضافه می‌شود، بدون اینکه کسی HTML را دستی ویرایش کند.

در این تمرین، بخش‌هایی از همان صفحه هفته اول را **زنده** می‌کنید؛ یعنی داده‌های آن را با `fetch` از API آزمایشی **دار و دسته چهل‌نفره (Gang of Forty)** می‌خوانید و با JavaScript روی صفحه می‌سازید.

این تمرین سطح‌بندی شده است. فقط **سطح ۱** برای تحویل لازم است و بقیه سطح‌ها اختیاری هستند. هر سطحی که کامل کنید، در Pull Request اعلام می‌کنید. هدف این است که همه بتوانند تمرین را تحویل دهند و هر کس که وقت و انرژی بیشتری دارد، تا سقف تمرین جلو برود.

- **سطح ۱ (حداقل تحویل):** یک کاروسل از روی `products.json`
- **سطح ۲:** کاروسل «پیشنهاد ویژه امروز» با موجودی و تخفیف
- **سطح ۳:** دسته‌بندی‌ها، منو، Header و Footer از روی داده
- **سطح ۴ (سقف تمرین):** کل صفحه از روی داده، به‌علاوه سبد خرید ماندگار

اگر صفحه هفته اول شما هنوز کامل نیست، نگران نباشید؛ برای این تمرین فقط کافی است صفحه شما حداقل یک کاروسل محصول داشته باشد.

این تمرین درس‌های هفته دوم را کنار هم قرار می‌دهد:

- Flexbox و Grid
- `localStorage` و URL Fragment
- توابع، آرایه‌ها و Objectها
- `querySelector`، `createElement` و رویدادها
- `fetch`، `Promise.all` و `async` / `await`

API فروشگاه

API آزمایشی در پوشه `api/` مخزن Gang of Forty قرار دارد و فقط‌خواندنی است. این فایل‌ها طوری طراحی شده‌اند که دقیقاً با اجزای صفحه هفته اول جور باشند:

- **`products.json`:** ۴۳ محصول. هر محصول `id`، `categoryId`، `name`، `emoji`، `unit`، `price` (به تومان) و `tags` دارد. هر دسته حداقل ۶ محصول دارد. ← **کارت‌های کاروسل**
- **`categories.json`:** ۷ دسته. هر دسته `id`، `slug`، `name`، `emoji` و `color` دارد. مقدار `slug` همان Fragment بخش است (`#dairy`) و `color` رنگ پس‌زمینه آیکون است. ← **دسته‌بندی‌ها، منوی «همه دسته‌ها» و لینک‌های Footer**
- **`stock.json`:** موجودی هر محصول در `items[id]`. موجودی `0` یعنی ناموجود. ← **حالت «ناموجود» کارت**
- **`offers.json`:** ۱۰ تخفیف. هر تخفیف `productId`، `percent` و `label` دارد. ← **برچسب تخفیف، قیمت خط‌خورده و کاروسل «پیشنهاد ویژه امروز»**
- **`shop.json`:** `name`، `slogan`، `notice` (نوار اطلاع‌رسانی)، `menu` (منوی اصلی)، `about`، `address`، `phone`، `schedule` (ساعت کاری)، `help` (لینک‌های راهنما) و `copyright`. ← **Header، منوی اصلی و Footer**
- **`home.json`:** `hero` و `side` (دو بنر بالای صفحه)، `promos` (بنرهای تبلیغاتی وسط صفحه) و `carousels` (فهرست کاروسل‌های صفحه: عنوان، Fragment و منبع کارت‌ها). ← **بنرها و ترتیب کاروسل‌ها**

ارتباط فایل‌ها با `id` است:

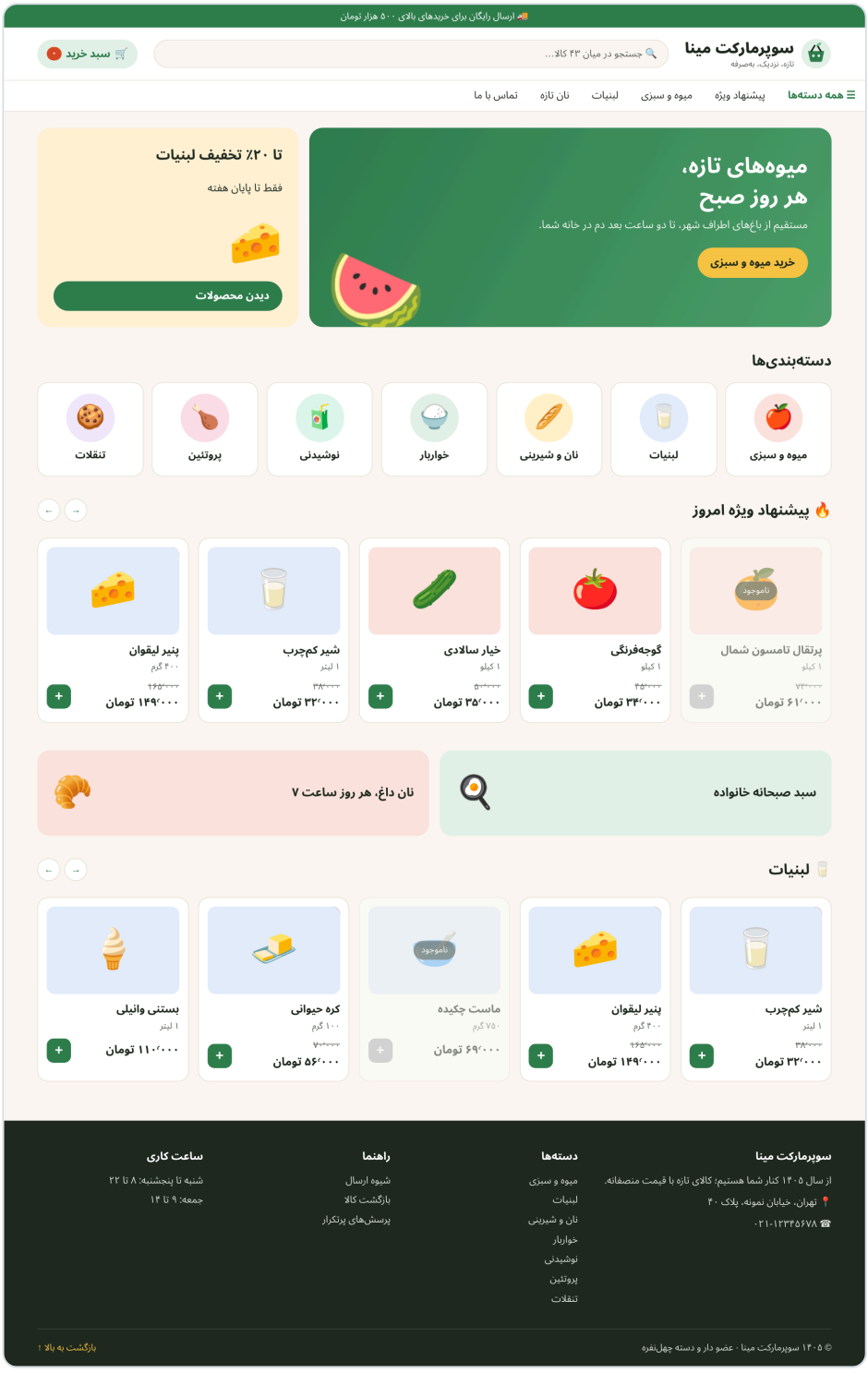
- `offer.productId` ↔ `stock.items[product.id]` ↔ `product.id`
- `category.id` ↔ `product.categoryId`

آدرس API:

- از صفحه فروشگاه خودتان (`shops/<slug>/index.html`): `../../api/products.json`
- روی سیستم خودتان، **ریشه مخزن** را با `npx serve .` یا Live Server اجرا کنید. اگر صفحه را با دابل‌کلیک (`file://`) باز کنید، مرورگر درخواست `fetch` را مسدود می‌کند.

نام فروشگاه، لوگو و رنگ‌های شما همچنان متعلق به خودتان است. لازم نیست نام «سوپرمارکت مینا» از `shop.json` را نمایش دهید؛ در سطح ۳ می‌توانید فقط بخش‌هایی از `shop.json` را استفاده کنید که با فروشگاه شما جور است.

نمای صفحه نمونه (سطح ۴)



همان صفحه هفته اول؛ این بار همه بخش‌ها از API ساخته شده‌اند: نوار اطلاع‌رسانی و Header از `shop.json`، بنرها از `home.json`، دسته‌بندی‌ها از `categories.json`، دو کاروسل از `products.json` و `stock.json` و `offers.json` و Footer دوباره از `shop.json`. فقط لوگوی SVG در HTML باقی مانده است.

سطح ۱: یک کاروسل زنده (حداقل تحویل)

یکی از کاروسل‌های صفحه هفته اول را انتخاب کنید، برای مثال «لبنیات»، و کارت‌های دستی آن را با داده API جایگزین کنید.

- محتوای کارت‌های آن کاروسل از HTML حذف شود؛ فقط ظرف خالی کاروسل در HTML بماند.
- فایل `products.json` با `fetch` و `await` / `async` خوانده شود.
- محصولات یک دسته با `filter` روی `categoryId` جدا شوند.
- برای هر محصول یک کارت با همان ظاهر هفته اول ساخته شود: آیکون، نام، مقدار یا وزن، قیمت به تومان با اعداد فارسی و دکمه افزودن.
- کارت‌ها با `createElement` و `textContent` ساخته شوند؛ داده API هرگز مستقیم داخل `innerHTML` قرار نگیرد.
- رفتار کاروسل هفته اول (اسکرول افقی، ایستادن روی ابتدای کارت، RTL و دکمه‌های ابتدا و انتها) همچنان کار کند.
- تا وقتی داده نرسیده، یک پیام یا Skeleton «در حال بارگذاری» دیده شود.
- اگر درخواست شکست خورد (`res.ok` برابر false یا قطع اینترنت)، به‌جای کاروسل خالی یک پیام خطای قابل فهم نمایش داده شود.

راهنمایی: قیمت فارسی با جداکننده هزارگان:

```
new Intl.NumberFormat('fa-IR').format(165000) // ۱۶۵٬۰۰۰
```

سطح ۲: پیشنهاد ویژه امروز

کاروسل «پیشنهاد ویژه امروز» را زنده کنید. این کاروسل به سه فایل نیاز دارد.

- فایل‌های `products.json`، `stock.json` و `offers.json` **هم‌زمان** با `Promise.all` خوانده شوند، نه پشت سر هم.
- کاروسل فقط محصولاتی را نشان دهد که در `offers.json` تخفیف دارند.
- کارت تخفیف‌دار: برچسب درصد تخفیف، قیمت قبلی خط‌خورده و قیمت جدید.
- کارت ناموجود (موجودی `0`): کارت کم‌رنگ، برچسب «ناموجود» و دکمه افزودن غیرفعال (`disabled`).
- کاروسل سطح ۱ هم از همین داده‌ها استفاده کند؛ اگر محصولی در آن کاروسل تخفیف دارد یا ناموجود است، همان ظاهر را بگیرد.
- برای پیدا کردن تخفیف و موجودی هر محصول، از `find` یا یک `Map` استفاده کنید.



کارت ناموجود (سمت راست) کم‌رنگ است و دکمه‌اش غیرفعال است. کارت‌های تخفیف‌دار قیمت قبلی خط‌خورده و قیمت جدید را نشان می‌دهند.

سطح ۳ و سطح ۴: کل صفحه از روی داده

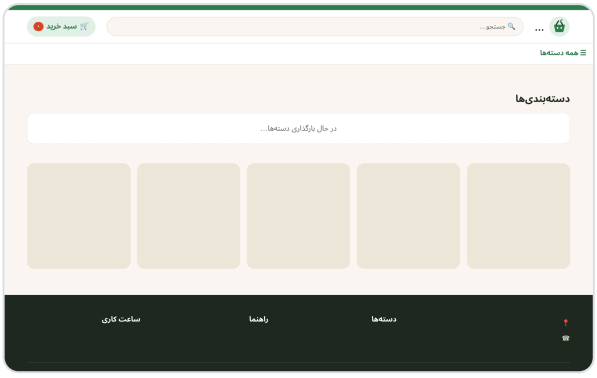
سطح ۳: دسته‌بندی‌ها، منو، Header و Footer

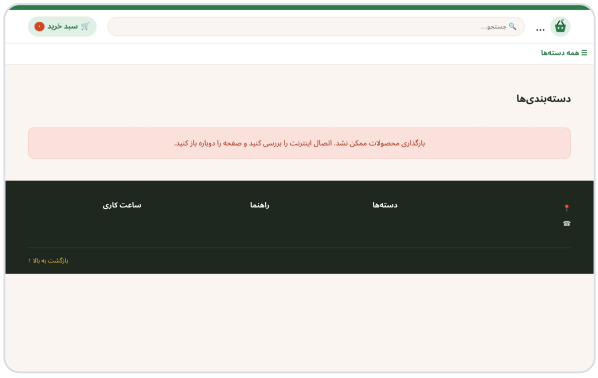
- بخش دسته‌بندی‌ها از `categories.json` ساخته شود: آیکون با رنگ `color`، نام و لینک به `#slug`.
- منوی «همه دسته‌ها» و لینک‌های دسته در Footer از همان داده ساخته شوند.
- نوار اطلاع‌رسانی، شعار، منوی اصلی، درباره ما، آدرس، تلفن، ساعت کاری و کپی‌رایت از `shop.json` خوانده شوند (فقط بخش‌هایی که با فروشگاه شما جور است).
- لینک Fragment هر دسته (برای مثال `#dairy`) همچنان کاربر را به بخش درست ببرد، حتی بعد از اینکه بخش‌ها با JavaScript ساخته شده‌اند.

سطح ۴: سقف تمرین

- بنرهای بالای صفحه و بنرهای تبلیغاتی از `home.json` ساخته شوند.
- کاروسل‌ها از فهرست `home.json` → `carousels` ساخته شوند؛ اگر یک کاروسل جدید به این فهرست اضافه شود، بدون تغییر HTML در صفحه ظاهر شود.
- همه فایل‌ها با **یک** `Promise.all` خوانده شوند.
- دکمه افزودن تعداد سبد خرید در Header را زیاد کند و این عدد در `localStorage` ذخیره شود؛ بعد از Reload صفحه باقی بماند.
- متن جعبه جستجو تعداد واقعی کالاها را نشان دهد (برای مثال «جستجو در میان ۴۳ کالا...»).
- در HTML فقط اسکلت صفحه، لوگوی SVG و ظرف‌های خالی باقی بماند.

حالت‌های بارگذاری و خطا





راست، در حال بارگذاری: تا رسیدن داده، به‌جای کارت‌ها Skeleton یا پیام «در حال بارگذاری» دیده می‌شود. **چپ، خطا:** اگر API در دسترس نباشد، کاربر یک پیام قابل فهم می‌بیند، نه یک صفحه خالی یا شکسته.

هر دو حالت را خودتان امتحان کنید:

- **بارگذاری:** در DevTools → Network، سرعت را روی **Slow 3G** بگذارید و صفحه را Reload کنید.
- **خطا:** آدرس API را موقتاً به یک فایل اشتباه (مثلاً `../../api/nope.json`) تغییر دهید، یا در DevTools → Network حالت **Offline** را فعال کنید.

قوانین و تحویل

قوانین

- از هیچ Framework یا کتابخانه‌ای (مثل jQuery یا axios) استفاده نشود؛ فقط JavaScript خود مرورگر.
- JavaScript در یک فایل جداگانه داخل پوشه خودتان باشد (مثلاً `shops/<slug>/app.js`)، نه داخل Attributeهایی مثل `onclick`.
- پوشه `api/` را **تغییر ندهید**؛ این پوشه بین هر ۴۰ فروشگاه مشترک است. فقط فایل‌های پوشه `shops/<slug>/` خودتان تغییر کنند.
- هیچ خطایی در Console دیده نشود.
- HTML صفحه باید از Validator پروژه با موفقیت عبور کند.

تحویل

یک Branch با نام `feat/<slug>-w2-api` بسازید و یک Pull Request به مخزن Gang of Forty ارسال کنید.

توضیح Pull Request باید شامل موارد زیر باشد:

۱. **سطح:** به کدام سطح رسیدید؟ چک‌لیست همان سطح را کپی کنید و موارد انجام‌شده را علامت بزنید.
۲. یک تصویر از صفحه و یک تصویر از DevTools → Network که درخواست‌های JSON در آن دیده می‌شوند.
۳. **حالت‌ها:** یک تصویر از حالت بارگذاری یا حالت خطای صفحه خودتان.
۴. **`fetch` و 404:** وقتی فایل وجود ندارد، آیا `fetch` خطا می‌دهد؟ کد شما این حالت را چطور تشخیص می‌دهد؟
۵. **`textContent` یا `innerHTML`:** چرا داده API را با `textContent` در صفحه قرار دادید؟
۶. (از سطح ۲ به بعد) **`Promise.all`:** چرا سه فایل را هم‌زمان خواندید و نه پشت سر هم؟ در Network چه تفاوتی دیده می‌شود؟

معیار انجام (Definition of Done)

این تمرین زمانی انجام‌شده محسوب می‌شود که:

- حداقل **سطح ۱** کامل شده باشد: یک کاروسل از روی `products.json` ساخته شود و حالت بارگذاری و خطا داشته باشد.
- رفتار کاروسل هفته اول (اسکرول، ایستادن روی کارت و RTL) همچنان کار کند.
- هیچ داده‌ای از API با `innerHTML` در صفحه قرار نگرفته باشد.
- هیچ خطایی در Console وجود نداشته باشد.
- Pull Request فقط پوشه خودتان را تغییر داده، از CI با موفقیت عبور کرده و به پرسش‌های بخش تحویل پاسخ داده است.

چالش اختیاری: با کلیک روی یک دسته، آدرس صفحه به `?cat=dairy` تغییر کند و فقط محصولات همان دسته نمایش داده شود (از `URLSearchParams` و `history.pushState` در درس سوم استفاده کنید). یا با `Promise.race`، اگر API بیشتر از ۳ ثانیه جواب نداد، پیام «اتصال کند است» نمایش دهید.